

Template - Modelo ADDIE (Etapa de Análise)

Propósito: Criação de Curso Inovador em Computação

Compreensão do Problema Educacional:

- Tema do curso escolhido:
Introdução à Computação / Pensamento Computacional para estudantes novatos de Pós-Graduação em Ciência da Computação
- Contexto (acadêmico, corporativo, híbrido, etc.):
Acadêmico (pós-graduação em computação).

👉 **Problema educacional que o curso pretende resolver:** (Descreva de forma clara e contextualizada a motivação para o curso)

A área de computação está cada vez mais interligada às outras áreas de conhecimento, sendo utilizada por elas em Ciência de Dados, Inteligência Artificial, Automação de Processos, Desenvolvimento de Software e Sistemas, entre outros. Porém, muitas vezes os alunos com formações em outras áreas não chegam na pós-graduação com o conhecimento necessário/esperado, e esse gap acaba afetando a permanência no curso e no desempenho acadêmico. O curso visa auxiliar os estudantes de Pós-Graduação em Ciência da Computação do CIn que não possuem formação acadêmica na área nos primeiros passos sobre pensamento computacional.

1. Caracterização do Público-Alvo

Objetivo: Compreender profundamente o aprendiz.

- Perfil (formação, nível, área):
 - Formação: graduação ou pós-graduação em qualquer área do conhecimento que não seja em Computação.
 - Nível: Cursando pós-graduação no CIn.
- Experiência prévia em computação:
Nenhuma.
- Motivações para aprender:
Aplicar os conhecimentos de TI à sua área de conhecimento, transição de carreira e maior probabilidade de sucesso durante a trajetória na pós-graduação no CIn.
- Dificuldades comuns:
Elementos sintáticos e lexicais, estruturas lógicas e fluxo dinâmico, depuração e fragilidade de código, fundamentos matemáticos e pensamento computacional.

- Contexto (tempo disponível, acesso a recursos, etc.):
Tempo limitado, dividido entre disciplinas, pesquisa e atividades profissionais; acesso a ferramentas digitais

👉 **Persona construída pela equipe:** (Descreva um aprendiz típico com nome fictício em 3–5 linhas)

João é um biólogo apaixonado por tecnologia que decidiu voltar à sala de aula. Agora, como estudante de Pós-Graduação em Ciência da Computação, ele tem um foco claro: aprender a usar a análise de dados e machine learning para inovar a forma como analisamos e entendemos o genoma.

2. Análise de Necessidades (Learning Gap)

Objetivo: Identificar a lacuna entre o estado atual e o desejado.

- O que os aprendizes já sabem (conhecimentos prévios):
Conhecimentos consolidados em sua área de formação.
- O que eles precisam saber/fazer ao final do curso:
Aplicar o pensamento computacional de forma independente, adquirindo a capacidade de traduzir e modelar problemas analíticos de seus domínios originais em passos lógicos de algoritmos e pseudo-códigos.
- Principais lacunas identificadas:
As principais lacunas de aprendizagem concentram-se na ausência de modelos mentais estruturados para o pensamento computacional, nas severas dificuldades técnicas com a sintaxe e a depuração de erros (debugging), além de barreiras socioemocionais marcadas pelo isolamento e pela dificuldade de enxergar a relevância prática da programação em seus domínios de origem.
- Evidências do gap (dados, relatos, experiências):
As evidências desse gap são comprovadas por relatos de profunda angústia emocional, pela paralisação completa das pesquisas independentes diante de falhas no código, por comportamentos de desengajamento acadêmico e pela forte insegurança em gerar scripts excessivamente improvisados e não confiáveis.

👉 **Síntese do Gap:** (Descreva em 2–3 linhas a principal lacuna de aprendizagem)

Embora possuam forte domínio teórico e estatístico em suas áreas de origem, falta aos alunos a base do pensamento computacional e de modelos mentais sobre a estrutura do código. Essa lacuna os impede de abstrair problemas em algoritmos estruturados, forçando o uso ineficiente de "tentativa e erro" e causando isolamento e paralisia diante de erros de execução (debugging), o que inviabiliza a autonomia técnica em suas pesquisas

3. Análise de Tarefas

Objetivo: Mapear o que o aprendiz precisa ser capaz de fazer.

- Tarefas-chave que o aprendiz deverá executar:

- Tarefa 1: Traduzir problemas analíticos e de pesquisa de seus domínios originais em etapas lógicas e pseudo-códigos.
- Tarefa 2: Estruturar e escrever códigos (em linguagens focadas em dados, como Python ou R), demonstrando compreensão clara da estrutura estática e do fluxo dinâmico de execução.
- Tarefa 3: ~~Testar e depurar (debugging) seus próprios scripts de forma autônoma, lidando ativamente com erros de execução e com a manutenção de códigos frágeis.~~
- Conhecimentos necessários:
Modelos mentais da estruturação de algoritmos, e fundamentos de pensamento computacional aplicados à análise de dados.
- Habilidades necessárias:
Capacidade de abstração cognitiva, pensamento analítico para resolução de problemas estruturados (substituindo o comportamento ineficiente de "tentativa e erro"), e habilidade para ler e interpretar códigos de terceiros (reconhecimento de padrões).
- Atitudes/comportamentos esperados:
Resiliência e tolerância à frustração perante falhas no código, proatividade para o trabalho colaborativo e troca de conhecimentos (mitigando o isolamento comum na área), e autoeficácia para superar os estereótipos de que a computação é apenas para especialistas tradicionais

👉 Complexidade das tarefas:

☐ Baixa ☐ Média ☒ Alta

Justifique:

As tarefas apresentam alta complexidade para o público-alvo, pois exigem uma profunda mudança cognitiva em direção ao pensamento computacional, indo muito além da simples memorização de sintaxe.

4. Análise Instrucional

Objetivo: Definir o tipo e nível de instrução necessários.

- Tipo de aprendizagem predominante:
☐ Conceitual ☐ Procedimental ☐ Atitudinal ☒ Mista
- Nível de profundidade necessário:
☒ Introdutório ☐ Intermediário ☐ Avançado
- Necessidade de prática:
☐ Baixa ☐ Moderada ☒ Alta
- Estratégias instrucionais mais adequadas (hipóteses):
☒ PBL ☒ Projetos ☒ Estudos de caso ☐ Simulação ☐ Aula expositiva ☒ Outra:
Inquérito Guiado (Guided Inquiry), Andaimos Cognitivos (Scaffolding) e Programação em Pares

👉 Justificativa da abordagem instrucional:

(Explique por que essas estratégias são adequadas ao contexto, considerando o perfil do aprendiz)

O uso do PBL (Aprendizagem Baseada em Problemas) e de Projetos é a estratégia mais recomendada para mitigar a principal causa de desengajamento: a falta de relevância percebida. Estudos de caso também são eficazes para demonstrar a aplicação direta de técnicas computacionais na resolução de problemas específicos de outras disciplinas. A inclusão de Inquérito Guiado (Guided Inquiry) e Andaimos Cognitivos

(Scaffolding) é vital para promover a transição cognitiva do aluno, fornecendo roteiros e perguntas reflexivas que os ajudam a interpretar e construir códigos de forma sistemática e menos frustrante. Por fim, estratégias colaborativas como a Programação em Pares são essenciais para combater ativamente os estereótipos de que a computação é um trabalho excessivamente solitário

5. Levantamento de Competências

Objetivo: Definir o que será desenvolvido no curso.

- Competências técnicas (**hard skills**):
 - Pensamento Computacional e Abstração.
 - Compreensão Estrutural de Código.
 - Programação Aplicada a Dados.
 - ~~Depuração (Debugging) e Resolução de Erros~~
- Competências socioemocionais (**soft skills**):
 - Resiliência e Tolerância à Frustração
 - Trabalho Colaborativo e Comunicação
 - Autoeficácia

Padrões de desempenho esperados:

(Como será considerado um desempenho adequado?)

O aprendiz será considerado apto quando for capaz de pegar uma base de dados real de sua própria pesquisa, modelar o problema logicamente (sem recorrer à tentativa e erro cega) e desenvolver, testar e depurar um script de forma autônoma. Um desempenho adequado também exige que o aluno consiga explicar o funcionamento e o fluxo do seu código para seus pares de forma clara, produzindo uma solução de software minimamente confiável e reproduzível.



Reflexão Final

Principal insight sobre o aprendiz:

O maior obstáculo para esse público não é apenas a memorização da sintaxe da linguagem, mas sim a ausência de modelos mentais estruturados para o pensamento computacional (compreensão da estrutura estática e do fluxo dinâmico do código). Além disso, eles enfrentam fortes barreiras socioemocionais, como isolamento e ansiedade, que os levam a recorrer à ineficiente estratégia de "tentativa e erro".

Principais implicações para o design do curso:

O curso deve afastar-se do ensino puramente expositivo e adotar uma abordagem de profunda transformação sociotécnica, indo além da transferência de sintaxe. O design instrucional precisa ser altamente contextualizado, ancorando os desafios em dados e problemas reais dos domínios de origem dos alunos (como o uso de PBL) para justificar a relevância da programação.

👉 Riscos caso a análise seja ignorada:

Se as necessidades socioemocionais e a falta de modelos mentais forem negligenciadas, os aprendizes sofrerão forte angústia emocional e desengajamento comportamental.

Marque se sua análise:

- ☐ Está centrada no aprendiz (não no conteúdo)
- ☐ Identifica claramente o gap de aprendizagem
- ☐ Considera contexto real de aplicação
- ☐ Integra competências técnicas e socioemocionais
- ☐ Evita soluções genéricas
- ☐ Gera insumos claros para o design do curso

Refletindo sobre essa atividade:

- O que aprendemos sobre o aprendiz que não sabíamos antes?
- O que foi mais difícil nesta análise?
- Como o trabalho em equipe contribuiu para a compreensão do problema?
- Como esta análise contribui para tornar o curso mais relevante, significativo e inovador?